

Gramáticas Livres de Contexto

Análise sintática • Compiladores e Interpretadores

Objetivo da aula

Compreender como uma gramática livre de contexto especifica a estrutura sintática de uma linguagem e como terminais, não-terminais, produções, derivações e árvores de derivação se relacionam ao parser do MINIC.

Projeto MINIC

Nesta aula, a equipe inicia a formalização da gramática que será usada na Etapa 2 — analisador sintático e AST.

Referência principal: Aho, Lam, Sethi e Ullman, Compiladores: Princípios, Técnicas e Ferramentas, 2ª ed., Capítulo 4, Seção 4.2, pp. 125–132.

1. Por que precisamos de gramáticas?

O analisador léxico transforma caracteres em tokens. O analisador sintático recebe essa sequência de tokens e verifica se ela respeita a estrutura da linguagem. Para descrever essa estrutura de maneira precisa, usamos uma gramática.

Uma gramática não apenas lista tokens permitidos: ela organiza esses tokens em construções hierárquicas, como expressões, comandos, declarações e funções. Essa estrutura será essencial para construir a árvore sintática abstrata (AST) do MINIC.

Exemplo de comando condicional:

```
stmt → if ( expr ) stmt else stmt
```

A produção acima indica que um comando pode ser formado pela palavra-chave `if`, seguida de uma expressão entre parênteses, um comando, a palavra-chave `else` e outro comando.

Conexão com o projeto: a gramática da Etapa 2 será a “especificação” que orientará o parser. Cada produção deve corresponder a uma construção válida do MINIC.

2. Definição formal de uma GLC

Uma gramática livre de contexto (GLC) é definida por quatro componentes:

Componente	Função	Exemplo no MINIC
Terminais	Símbolos básicos da entrada, normalmente nomes de tokens.	<code>if</code> , <code>(</code> , <code>)</code> , <code>ID</code> , <code>INT</code> , <code>;</code>
Não-terminais	Variáveis sintáticas que representam conjuntos de cadeias.	<code>stmt</code> , <code>expr</code> , <code>decl</code>
Símbolo inicial	Não-terminal a partir do qual os programas são gerados.	<code>program</code>
Produções	Regras que definem como combinar terminais e não-terminais.	<code>stmt → if (expr) stmt</code>

A palavra “livre de contexto” significa que uma produção pode substituir um não-terminal independentemente dos símbolos que aparecem ao seu redor. O não-terminal é reescrito conforme as regras da gramática, sem depender de um contexto específico.

3. Exemplo de gramática para expressões

A gramática a seguir descreve expressões aritméticas simples com identificadores, parênteses e os operadores +, −, × e ÷:

```
E → E + T | E − T | T  
T → T * F | T / F | F  
F → ( E ) | id
```

Leitura: uma expressão (E) pode ser uma soma ou subtração de termos, ou simplesmente um termo. Um termo (T) pode ser um produto ou divisão de fatores, ou simplesmente um fator. Um fator (F) pode ser uma expressão entre parênteses ou um identificador.

A organização em níveis permite representar a precedência: multiplicação e divisão ficam dentro de T, enquanto soma e subtração ficam dentro de E.

Imagem de apoio do livro

- falando apenas sobre o nome do token. Assumimos que os terminais são os primeiros componentes dos tokens gerados pelo analisador léxico. Em (4.4), os terminais são as palavras-chave **if** e **else**, e os símbolos “(” e “)”.
2. *Não-terminais* são variáveis sintáticas que representam conjuntos de cadeias. Em (4.4), *stmt* e *expr* são não-terminais. Os conjuntos de cadeias denotando não-terminais auxiliam na definição da linguagem gerada pela gramática. Os não-terminais impõem uma estrutura hierárquica sobre a linguagem que é a chave para a análise sintática e tradução.
 3. Em uma gramática, um não-terminal é distinguido como o *símbolo inicial*, e o conjunto de cadeias que ele representa é a linguagem gerada pela gramática. Por convenção, as produções para o símbolo inicial são listadas primeiro.
 4. As produções de uma gramática especificam a forma como os terminais e os não-terminais podem ser combinados para formar cadeias. Cada *produção* consiste em:
 - (a) Um não-terminal chamado de *cabeça* ou *lado esquerdo* da produção; essa produção define algumas das cadeias representadas pela cabeça.
 - (b) O símbolo $\rightarrow$. Às vezes, $::=$ é usado no lugar da seta.
 - (c) Um *corpo* ou *lado direito da produção* consiste em zero ou mais terminais e não-terminais. Os componentes do corpo descrevem uma forma como as cadeias do não-terminal do lado esquerdo da produção podem ser construídas.

EXEMPLO 4.5: A gramática na Figura 4.2 define expressões aritméticas simples. Nessa gramática, os símbolos terminais são

id + - * / ()

Os símbolos não-terminais são *expression*, *term* e *factor*, e *expression* é o símbolo inicial da gramática.

<i>expression</i>	$\rightarrow$	<i>expression</i> + <i>term</i>
<i>expression</i>	$\rightarrow$	<i>expression</i> - <i>term</i>
<i>expression</i>	$\rightarrow$	<i>term</i>
<i>term</i>	$\rightarrow$	<i>term</i> * <i>factor</i>
<i>term</i>	$\rightarrow$	<i>term</i> / <i>factor</i>
<i>term</i>	$\rightarrow$	<i>factor</i>
<i>factor</i>	$\rightarrow$	(<i>expression</i>)
<i>factor</i>	$\rightarrow$	id

FIGURA 4.2 Gramática para expressões aritméticas simples.

4.2.2 CONVENÇÕES DE NOTAÇÃO

Para evitar ter de dizer sempre “estes são os terminais”, “estes são os não-terminais” etc., adotaremos as seguintes convenções de notação para as gramáticas no restante deste livro.

1. Estes símbolos são terminais:
 - (a) Letras minúsculas do início do alfabeto, como *a*, *b*, *c*.
 - (b) Símbolos operadores como +, * e assim por diante.
 - (c) Símbolos de pontuação como parênteses, vírgula e assim por diante.
 - (d) Os dígitos 0, 1, ..., 9.
 - (e) Cadeias em negrito como **id** ou **if**, cada um representando um único símbolo terminal.
2. Estes símbolos são não-terminais:
 - (a) Letras maiúsculas do início do alfabeto, como *A*, *B*, *C*.
 - (b) A letra *S*, que, quando aparece, normalmente é o símbolo inicial da gramática.
 - (c) Nomes em minúsculas e itálico, como *expr* ou *stmt*.
 - (d) Nas construções das linguagens de programação, letras maiúsculas podem ser usadas para representar não-terminais. Por exemplo, os não-terminais para expressões, termos e fatores normalmente são representados por *E*, *T* e *F*, respectivamente.
3. Letras maiúsculas do fim do alfabeto, como *X*, *Y*, *Z*, representam símbolos da gramática, ou seja, não-terminais ou terminais.
4. Letras maiúsculas do fim do alfabeto, principalmente *u*, *v*, ..., *z*, representam cadeias de terminais, possivelmente vazias.
5. Letras gregas minúsculas, por exemplo α , β , γ , representam cadeias de símbolos da gramática, possivelmente vazias. Assim, uma produção genérica pode ser escrita como $A \rightarrow \alpha$, onde *A* é o lado esquerdo da produção ou a cabeça, e α representa o corpo ou lado direito da produção.

Figura extraída da página do livro correspondente à definição formal de gramática e ao exemplo de expressões aritméticas. Referência impressa: p. 126; PDF: p. 138.

4. Convenções de notação

Para escrever gramáticas de forma compacta, adotamos convenções. Em geral, letras maiúsculas e nomes em itálico representam não-terminais; símbolos de operadores, pontuação e nomes de tokens representam terminais.

Uma mesma cabeça de produção pode reunir alternativas usando a barra vertical:

```
E → E + T | E - T | T
```

Essa escrita é equivalente a três produções:

```
E → E + T  
E → E - T  
E → T
```

Atenção: a primeira produção normalmente indica o símbolo inicial. Para evitar ambiguidades no projeto, declare explicitamente o símbolo inicial `program` na documentação da gramática MINIC.

5. Derivações

Uma derivação é uma sequência de aplicações de produções. Começamos pelo símbolo inicial e substituímos um não-terminal pelo corpo de uma produção. O símbolo $\Rightarrow$ representa uma derivação em um passo; $\Rightarrow^*$ representa zero ou mais passos.

Exemplo simplificado:

```
E  $\Rightarrow$  E + T  $\Rightarrow$  T + T  $\Rightarrow$  id + T  $\Rightarrow$  id + id
```

A cadeia final contém apenas terminais e é chamada de sentença da gramática. Cadeias intermediárias podem conter terminais e não-terminais e são chamadas de formas sentenciais.

6. Derivações mais à esquerda e mais à direita

Em cada passo, pode haver mais de um não-terminal disponível para substituição. Duas estratégias importantes são:

Derivação mais à esquerda: sempre substitui o não-terminal mais à esquerda da forma sentencial. É frequentemente associada à construção top-down.

Derivação mais à direita: sempre substitui o não-terminal mais à direita. A análise bottom-up está relacionada à derivação mais à direita em ordem reversa.

Essas escolhas podem gerar sequências diferentes de substituição, mas a mesma sentença final quando a gramática é adequada.

Imagem de apoio do livro

As cadeias $E, -E, -(E), \dots, -(\text{id} + \text{id})$ são todas formas sentenciais dessa gramática. Escrevemos $E \xRightarrow{*} -(\text{id} + \text{id})$ para indicar que $-(\text{id} + \text{id})$ pode ser derivado de E .

Em cada passo de uma derivação, existem duas escolhas a serem feitas. Precisamos escolher qual não-terminal substituir e, tendo feito essa escolha, devemos selecionar uma produção com esse não-terminal como cabeça. Por exemplo, a derivação alternativa de $-(\text{id} + \text{id})$ a seguir difere da derivação (4.8) nos dois últimos passos:

$$E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E + E) \Rightarrow -(E + \text{id}) \Rightarrow -(\text{id} + \text{id}) \quad (4.9)$$

Cada não-terminal é substituído pelo mesmo corpo nas duas derivações, mas a ordem das substituições é diferente.

Para entender como os analisadores sintáticos funcionam, vamos considerar as derivações em que o não-terminal a ser substituído em cada passo é escolhido da seguinte maneira:

1. Em derivações *mais à esquerda*, o não-terminal mais à esquerda em cada forma sentencial sempre é escolhido. Se $\alpha \Rightarrow \beta$ é um passo em que o não-terminal mais à esquerda em α é substituído, escrevemos $\alpha \xRightarrow{lm} \beta$.
2. Em derivações *mais à direita*, o não-terminal mais à direita sempre é escolhido; escrevemos $\alpha \xRightarrow{rm} \beta$ nesse caso.

A derivação (4.8) corresponde à derivação mais à esquerda, e por isso pode ser reescrita como

$$E \xRightarrow{lm} -E \xRightarrow{lm} -(E) \xRightarrow{lm} -(E + E) \xRightarrow{lm} -(\text{id} + E) \xRightarrow{lm} -(\text{id} + \text{id})$$

Observe que (4.9) é uma derivação mais à direita.

Usando nossas convenções de notação, cada passo mais à esquerda pode ser escrito como $wA\gamma \xRightarrow{lm} w\delta\gamma$, onde w consiste apenas em terminais, $A \rightarrow \delta$ é a produção aplicada, e γ é uma cadeia de símbolos da gramática. Para enfatizar que α deriva β usando uma derivação mais à esquerda, escrevemos $\alpha \xRightarrow{*lm} \beta$. Se $S \xRightarrow{*lm} \alpha$, então dizemos que α é uma *forma sequencial mais à esquerda* da gramática dada.

Definições semelhantes são mantidas para derivações mais à direita. As derivações mais à direita às vezes são chamadas de derivações *canônicas*.

4.2.4 ÁRVORES DE DERIVAÇÃO E DERIVAÇÕES

Uma árvore de derivação é uma representação gráfica de uma derivação que filtra a ordem na qual as produções são aplicadas para substituir não-terminais. Cada nó interior de uma árvore de derivação representa a aplicação de uma produção. O nó interior é rotulado com o não-terminal A do lado esquerdo da produção; os filhos do nó são rotulados, da esquerda para a direita, pelos símbolos do corpo da produção pelo qual esse A foi substituído durante a derivação.

Por exemplo, a árvore de derivação para $-(\text{id} + \text{id})$, na Figura 4.3, resulta da derivação (4.8), bem como da derivação (4.9).

As folhas de uma árvore de derivação são rotuladas pelos não-terminais ou terminais e, lidas da esquerda para a direita, constituem uma forma sentencial, chamada *yield* ou *fronteira* da árvore.

Para ver o relacionamento entre derivações e árvores de derivação, considere qualquer derivação $\alpha_1 \Rightarrow \alpha_2 \Rightarrow \dots \Rightarrow \alpha_n$, onde α_1 é um único não-terminal A . Para cada forma sentencial α_i da derivação, é possível construir uma árvore de derivação cuja *yield* seja α_i . O processo é uma indução em i .

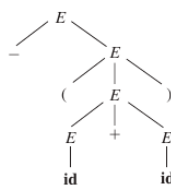


FIGURA 4.3 Árvore de derivação para $-(\text{id} + \text{id})$.

BASE: A árvore para $\alpha_1 = A$ é um único nó rotulado com A .

INDUÇÃO: Suponha que já tenhamos construído uma árvore de derivação com *yield* $\alpha_{i-1} = X_1 X_2 \dots X_k$ (observe que, de acordo com nossas convenções de notação, cada símbolo da gramática X_i é um não-terminal ou um terminal). Suponha que α_i seja derivado de α_{i-1} substituindo X_j , um não-terminal, por $\beta = Y_1 Y_2 \dots Y_m$. Ou seja, no i -ésimo passo da derivação, a produção $X_j \rightarrow \beta$ é aplicada a α_{i-1} para derivar $\alpha_i = X_1 X_2 \dots X_{j-1} \beta X_{j+1} \dots X_k$.

7. Árvores de derivação

Uma árvore de derivação representa graficamente a aplicação das produções. O nó interno é um não-terminal; seus filhos, da esquerda para a direita, correspondem aos símbolos do corpo da produção aplicada. As folhas formam a fronteira (ou yield) da árvore.

As árvores de derivação são uma ponte direta para a AST. A árvore de derivação mantém todos os detalhes da gramática; a AST remove elementos puramente sintáticos, como alguns parênteses e nós intermediários, preservando a estrutura relevante para a análise semântica e a geração de código.

Comparação conceitual:

Árvore de derivação	AST
Representa cada passo e símbolo da gramática.	Representa a estrutura essencial do programa.
Útil para explicar parsing e derivações.	Útil para semântica, IR e geração de código.

Imagem de apoio do livro



Para modelar esse passo da derivação, encontre a j -ésima folha a partir da esquerda na árvore de derivação corrente. Essa folha é rotulada com X_j . Dê a essa folha m filhos, rotulados como $Y_1, Y_2, \dots, Y_m$, a partir da esquerda. Como um caso especial, se $m = 0$, então $\beta = \epsilon$, e atribuímos à j -ésima folha um filho rotulado com ϵ .

EXEMPLO 4.10: A sequência de árvores de derivação construída a partir da derivação (4.8) é mostrada na Figura 4.4. No primeiro passo da derivação, $E \Rightarrow -E$. Para modelar esse passo, acrescente dois filhos, rotulados com $-$ e E , à raiz E da árvore inicial. O resultado é a segunda árvore.

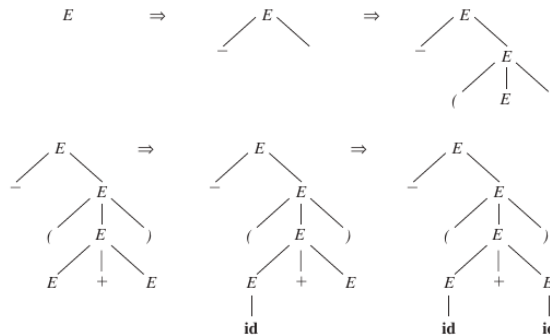


FIGURA 4.4 Sequência de árvores de derivação para a derivação (4.8).

No segundo passo da derivação, $-E \Rightarrow -(E)$. Consequentemente, inclua três filhos, rotulados com $($, E e $)$, à folha rotulada com E da segunda árvore, para obter a terceira árvore com fronteira $-(E)$. Continuando dessa forma, obtemos a árvore de derivação completa como a sexta árvore.

Como uma árvore de derivação ignora variações na ordem em que os símbolos nas formas sentenciais são substituídos, existe um relacionamento *muitos-para-um* entre as derivações e as árvores de derivações. Por exemplo, as derivações (4.8) e (4.9) são associadas à mesma árvore de derivação final ilustrada na Figura 4.4.

No texto a seguir, freqüentemente faremos a análise sintática utilizando uma derivação mais à esquerda ou mais à direita, uma vez que há um relacionamento *um-para-um* entre as árvores de derivação e as derivações mais à esquerda ou mais à direita. Ambas as derivações escolhem uma ordem particular para substituir os símbolos nas formas sentenciais, de modo que também filtram variações na ordem. Não é difícil mostrar que toda árvore de derivação possui associada a ela uma única derivação mais à esquerda e uma única derivação mais à direita.

4.2.5 AMBIGÜIDADE

De acordo com a Seção 2.2.4, uma gramática que produz mais de uma árvore de derivação para alguma sentença é considerada *ambígua*. Colocando de outra forma, uma gramática livre de contexto é ambígua se permitir a construção de mais de uma derivação mais à esquerda ou mais de uma derivação mais à direita para a mesma sentença.

EXEMPLO 4.11: A gramática de expressão aritmética (4.3) permite duas derivações mais à esquerda distintas para a sentença $\text{id} + \text{id} * \text{id}$:

$$\begin{array}{ll}
 E \Rightarrow E + E & E \Rightarrow E * E \\
 \Rightarrow \text{id} + E & \Rightarrow E + E * E \\
 \Rightarrow \text{id} + E * E & \Rightarrow \text{id} + E * E \\
 \Rightarrow \text{id} + \text{id} * E & \Rightarrow \text{id} + \text{id} * E \\
 \Rightarrow \text{id} + \text{id} * \text{id} & \Rightarrow \text{id} + \text{id} * \text{id}
 \end{array}$$

A Figura 4.5 mostra as duas árvores de derivação correspondentes.

Observe que a árvore de derivação da Figura 4.5(a) reflete a precedência normalmente assumida dos operadores $+$ e $*$, o que não acontece na árvore da Figura 4.5(b). Ou seja, é comum tratar o operador $*$ como tendo maior precedência do que $+$, o que corresponde à avaliação de uma expressão do tipo $a + b * c$ como $a + (b * c)$, em vez de $(a + b) * c$.



8. Aplicação ao MINIC

Para a Etapa 2, a equipe deve construir uma gramática que descreva pelo menos a estrutura geral de um programa, declarações, comandos e expressões. Um ponto de partida didático é:

```
program → declaration-list  
declaration-list → declaration declaration-list | ε  
declaration → var-declaration | fun-declaration  
var-declaration → type ID ;  
stmt → expr-stmt | compound-stmt | if-stmt | while-stmt | return-stmt  
expr-stmt → expr ; | ;
```

A gramática acima é apenas um esqueleto. A equipe deverá completá-la conforme a especificação oficial do MINIC, incluindo tipos, parâmetros, chamadas de função, vetores e precedência de operadores.

Checklist da aula

- ☐ Identificar terminais e não-terminais.
- ☐ Definir o símbolo inicial.
- ☐ Escrever as produções da linguagem.
- ☐ Testar derivações de programas válidos.
- ☐ Procurar ambiguidades.
- ☐ Registrar decisões de projeto para a Etapa 2.

9. Atividades

Atividade 1 — Classificação: para a gramática $E \rightarrow E + T \mid T$, $T \rightarrow \text{id}$, classifique E , T , $+$ e id como terminais ou não-terminais.

Atividade 2 — Derivação: usando a gramática $E \rightarrow E + T \mid T$, $T \rightarrow \text{id}$, mostre uma derivação para $\text{id} + \text{id}$.

Atividade 3 — MINIC: proponha produções para um comando de atribuição e para uma declaração de variável. Indique quais símbolos são tokens retornados pelo scanner.

Atividade 4 — Diagnóstico: encontre uma possível ambiguidade em uma gramática de if-then-else e explique por que ela é problemática para o parser.

10. Síntese e leitura orientada

Uma gramática livre de contexto fornece uma descrição formal da sintaxe. Seus terminais correspondem à entrada do parser; seus não-terminais organizam construções; as produções definem as combinações possíveis; e o símbolo inicial representa a linguagem completa.

As derivações mostram como uma sentença pode ser gerada. As árvores de derivação tornam essa estrutura visual e preparam o caminho para a construção da AST. Na próxima aula, a gramática do MINIC será analisada com mais detalhe, incluindo comandos, funções, precedência e associatividade.

Leitura principal

Aho, A. V.; Lam, M. S.; Sethi, R.; Ullman, J. D. Compiladores: Princípios, Técnicas e Ferramentas. 2ª edição. Capítulo 4 — Análise sintática: Seção 4.1, pp. 122–124; Seção 4.2, pp. 125–132; especialmente a definição formal, as convenções de notação, as derivações e as árvores de derivação.

Referência das imagens

As imagens desta apostila são reproduções de páginas do livro fornecido para esta atividade e aparecem exclusivamente como apoio didático, com indicação da paginação impressa. Livro: Compiladores: Princípios, Técnicas e Ferramentas, Aho, Lam, Sethi e Ullman, 2ª ed.

Fim da Aula 6